CI/CD with Jenkins
Lab 2
Working with a .NET Core CLI app

1 Executing .NET development workflow on Linux

Ensure that you are logged into your assigned user account on the EC2 Linux server.

Switch over to the project root folder for the .NET Core CLI app in your home directory.

```
cd githubprojects/PortableCliApp
```

IMPORTANT:
In the commands below, the \ acts as a line terminator in the Bash shell to allow a long command to be broken over several lines. You can remove it if you place all portions of the command onto a single line (using a note editor) before copying and pasting into the Bash shell.

```
dotnet --info
```

```
dotnet restore PortableCliApp.slnx
```

Restores all NuGet packages required by the projects in PortableCliApp.slnx and generates the dependency information needed for building.

```
dotnet build PortableCliApp.slnx --configuration Release --no-restore
```

Compiles all applicable projects in the solution.

- --configuration Release builds using the optimized Release configuration.
- --no-restore skips NuGet restore because it was already performed separately.

```
dotnet test PortableCliApp.slnx --configuration Release --no-build
```

Runs all test projects in the solution.

- --configuration Release tests the Release build.
- --no-build skips compilation and uses the binaries produced by the previous build command.

Because --no-build also implies --no-restore, neither restore nor build is repeated.

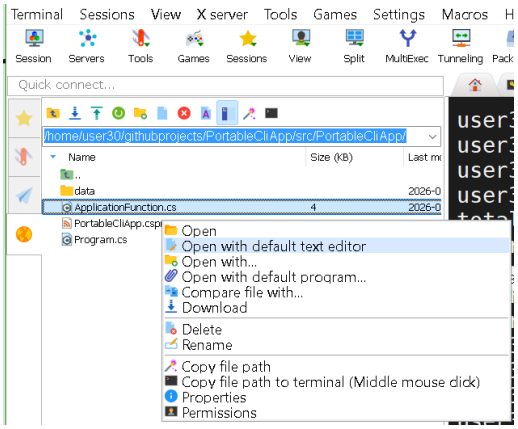
The unit tests should complete successfully the first time.

Simulate an error in the unit tests by intentionally introducing some errors into the application source code by editing: `src/PortableCliApp/ApplicationFunction.cs`

You can do this by using the built-in Linux nano editor:

```
nano src/PortableCliApp/ApplicationFunction.cs
```

or using the default MobaXTerm editor



You can also edit this file using an IDE (VS Code) in the project root folder on your local machine, and then transfer it to the EC2 Linux server by dragging and dropping into the navigation pane of the system

After introducing the errors, build the app and run the unit tests again to see how test errors are flagged:

```
dotnet build PortableCliApp.slnx --configuration Release --no-restore

dotnet test PortableCliApp.slnx --configuration Release --no-build
```

Correct the errors you introduced earlier by editing `ApplicationFunction.cs` again and then build the app and run the unit tests again to verify that they complete successfully.

```
dotnet build PortableCliApp.slnx --configuration Release --no-restore

dotnet test PortableCliApp.slnx --configuration Release --no-build
```

```
dotnet run --project src/PortableCliApp/PortableCliApp.csproj \
    --configuration Release --no-build
```

Starts the CLI application defined by the specified project.

- --project identifies the .csproj file to run.
- --configuration Release runs the Release version.
- --no-build uses the existing compiled output instead of rebuilding it.

```
dotnet publish src/PortableCliApp/PortableCliApp.csproj \
    --configuration Release --output artifacts/publish
```

Builds and prepares the application for deployment or distribution.

- The .csproj path identifies the project to publish.
- --configuration Release creates an optimized Release build.
- --output artifacts/publish places all published files in the specified directory.

Because --no-build is not specified, this command may restore and build the project again as required.

```
dotnet artifacts/publish/PortableCliApp.dll
```

Uses the .NET host to execute the published application DLL.
This works for a framework-dependent deployment, meaning a compatible .NET runtime must already be installed on the machine.

```
dotnet clean PortableCliApp.slnx --configuration Release
```

Runs the MSBuild clean target for all applicable projects and removes files generated by the Release build, mainly from locations such as:
`bin/Release/`
`obj/Release/`
It does not necessarily remove custom directories such as artifacts/publish unless the project's MSBuild configuration explicitly includes them in the clean process.

```
rm -rf artifacts
```

```
find . -depth -type d \
  \( -name bin -o \
     -name obj -o \
     -name TestResults -o \
     -name artifacts -o \
     -name .vs \) \
  -exec rm -rf -- {} +
```

2 Executing .NET development workflow on Windows (optional)

If you have installed .NET 10 on your Windows machine, you can also go through the complete workflow here as well. The commands are nearly identical to the case of Linux with the exception of the directory separator and line terminator characters (which are different for PowerShell compared to the Bash shell)

Open a PowerShell terminal in the project root folder for the .NET Core CLI app: `PortableCliApp`

IMPORTANT:
In the commands below, the ` acts as a line terminator in PowerShell to allow a long command to be broken over several lines. You can remove it if you place all portions of the command onto a single line (using a note editor) before copying and pasting into PowerShell.

```
dotnet --info
```

```
dotnet restore PortableCliApp.slnx
```

```
dotnet build PortableCliApp.slnx --configuration Release --no-restore
```

```
dotnet test PortableCliApp.slnx --configuration Release --no-build
```

The unit tests should complete successfully the first time.

Intentionally introduce some errors into the application source code by editing: `src\PortableCliApp\ApplicationFunction.cs`

You can do this by using a suitable IDE like VS Code or Notepad++

After introducing the errors, build the app and run the unit tests again to see how test errors are flagged:

```
dotnet build PortableCliApp.slnx --configuration Release --no-restore
```

```
dotnet test PortableCliApp.slnx --configuration Release --no-build
```

Correct the errors you introduced earlier by editing `ApplicationFunction.cs` again and then build the app and run the unit tests again to verify that they complete successfully.

```
dotnet build PortableCliApp.slnx --configuration Release --no-restore

dotnet test PortableCliApp.slnx --configuration Release --no-build



dotnet run --project src\PortableCliApp\PortableCliApp.csproj `
    --configuration Release --no-build


dotnet publish src/PortableCliApp/PortableCliApp.csproj `
    --configuration Release --output artifacts/publish


dotnet artifacts/publish/PortableCliApp.dll



dotnet clean PortableCliApp.slnx --configuration Release


Get-ChildItem -Path . -Directory -Recurse -Force |
    Where-Object { $_.Name -in @('bin', 'obj', 'TestResults',
'artifacts', '.vs') } |
    Sort-Object { $_.FullName.Length } -Descending |
    Remove-Item -Recurse -Force
```

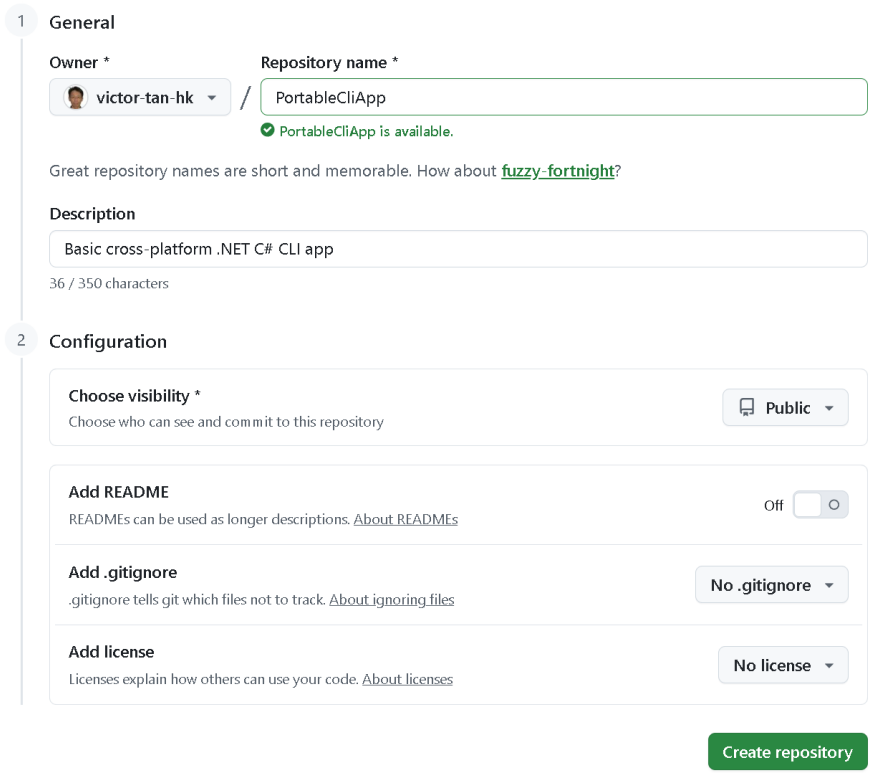
You may have to run the previous PowerShell command twice for it to take effect.

3 Creating a GitHub repo for the project

Login to your GitHub account.
Create a new repository and give it the same name as the Java project that we have been working with so far: `PortableCliApp`

Provide a simple description: `Basic cross-platform .NET C# CLI app`

Leave the Configuration details to the default settings as shown below:



Click Create repository.

Make sure you are logged into your assigned user account on the EC2 server and are currently in the project root folder for the .NET Core CLI app: `githubprojects/PortableCliApp`

If not, you can change to it with:

```
cd ~/githubprojects/PortableCliApp
```

Git is already installed on the server.
First, we check that this folder is not a proper Git repo yet with

```
git status
```

An error should indicate that this project folder is not yet a Git repo.

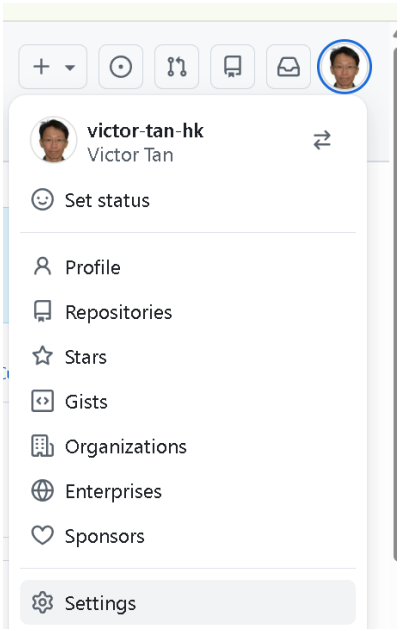
Next initialize it as a Git repo with all the files in the project folder with the following sequence of command in the Git Bash shell (you can type them in one at a time):

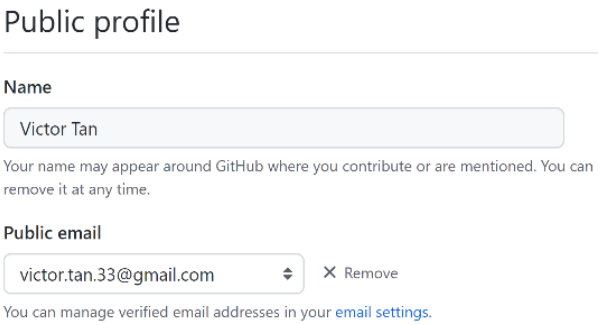
```
git init

git add .

git status
```

Before creating the first commit in this newly initialized repository, you need provide an identify for the individual creating the commit (the commit author). Since we are going to be pushing our local repo to populate the GitHub repo, we need to specify a correct GitHub identity (name and email) so

that the commit contents are pushed from the local repo have their authorship / origin correctly identified.

Go to your profile picture on the upper right hand corner of your main GitHub page, and access Settings: https://github.com/settings/profile





and note down the name and email address there. Use them in the configuration commands below:

```
git config --global user.name "your name"

git config --global user.email "your email address"
```

IMPORTANT:
We are configuring these details at the global level (`--global`) so that they apply only to the user account that you are logged in with, rather than at the host machine level (`--system`) since in this workshop we have multiple user accounts for different participants who will be accessing their own repos with their respective names and email addresses.

Double check that these configuration variable values have been set properly at the local level, which will be in the file `~/.gitconfig` in the current working directory

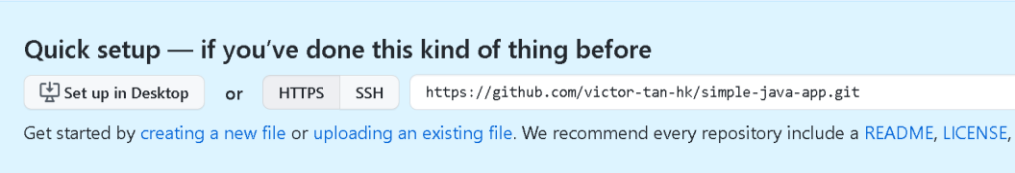
```
git config --list
```

Then we can create our first commit.

```
git commit -m "Added all project files in first commit"
```

```
git status
```

Return to the main page of your new GitHub repo that you just created. Copy the HTTPS URL shown there:



We will refer to this URL as *`remote-url`* in the commands to follow.

Back in the Bash shell in `simple-java-app`, and type:

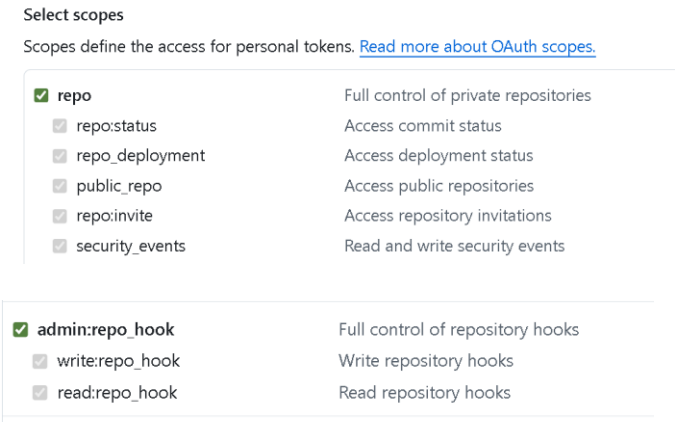
```
git remote add origin remote-url
```

Check that the origin handle is set to point to the correct repo URL with:

```
git remote --verbose
```

Before you can push the contents of the newly initialize repo to your remote GitHub repo, you must first obtain a classic personal access token (PAT):

For Select scopes, select the following 2 checkboxes:



When done, click Generate Token.

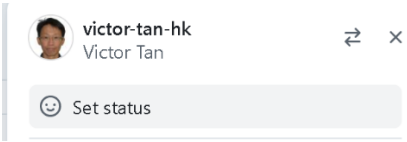
Once the token generation page appears, make sure you copy down the PAT to NotePad++ or somewhere where you can access it later before you navigate away from the page.

Finally, push the entire contents of the local repo (including all its branches) to the remote repo with:

```
git push -u origin --all
```

You will be prompted for your username and password:

Note: Your username (in this case, `victor-tan-hk`) may be different from your actual name (Victor Tan) associated with the GitHub account.



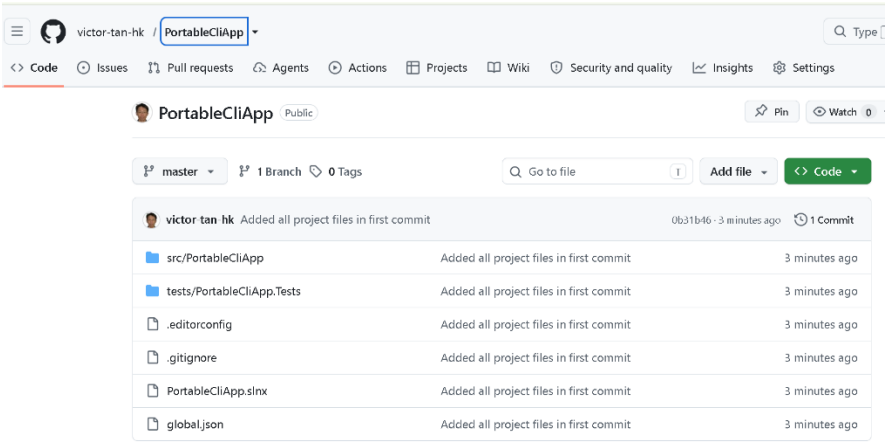

```
Username for 'https://github.com': yourusername
Password for 'https://githubaccountname@github.com': PAT
```

Copy and paste in the PAT from where you saved it earlier. Be very careful to do this slowly and correctly.

Assuming you have entered the correct GitHub user account name and PAT, the command should succeed with appropriate success messages in the Git Bash shell.

```
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
….
…
branch 'master' set up to track 'origin/master'.
```

Refresh the main repo page, you should see the contents that you have just pushed up.



Back in the Git Bash shell, type the following commands to verify that the remote tracking branches have been set up and that the local and upstream master are both in sync with each other.

```
git remote show origin

git branch -vv

git status
```

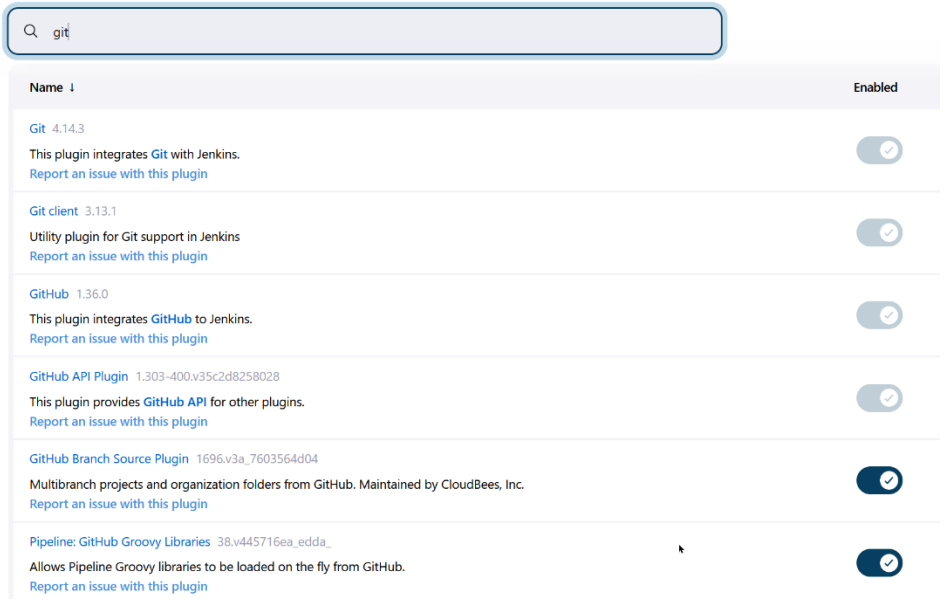
Other users (for e.g. team members of a project team) can now clone this remote repo to a local repo on their local machines and work on it via a branching workflow.

4 Checking / installing Git / GitHub plugins for Jenkins

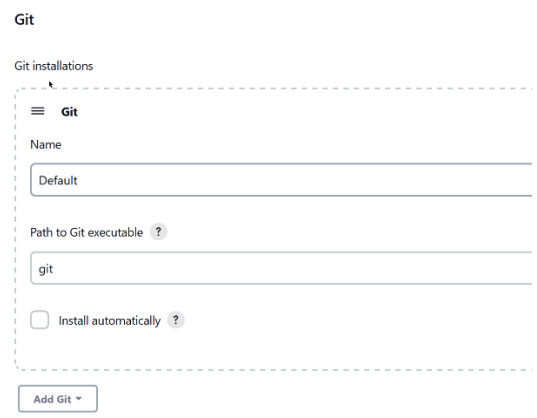
There are a variety of Git related plugins for Jenkins, which should have already being installed during the installation of Jenkins itself if you had selected the Install Suggested Plugins in the installation process itself.

To check, go to the main Jenkins Dashboard, Manage Jenkins -> Plugins -> Installed Plugins and search for the term: `git`

These are the plugins that should be installed which we will be using in subsequent labs. If they do not show up in your list, then go to Available Plugins and install them in the same way we have done previously.



You can initially configure Git in Manage Jenkins -> Tools, to ensure that the correct version of Git is used by Jenkins.



Typically, you can just use the default installation of Git (whether Linux / Windows), which should already be immediately accessible for standard installations of Git (via the PATH environment variable).

5 Creating a basic Jenkins job with declarative pipeline script

Ensure that you are logged into your assigned account on the Jenkins server / controller.

From the main Jenkins dashboard, create a new Item and name it: `userx-PortableCliApp`
Make this a Pipeline and select Ok.

You will be transitioned to the Configure section.
Select the Pipeline section.

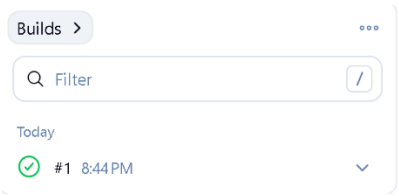
Copy and paste into the Script window this script from the `jenkinsfile collection` folder: `dotnetcli-jenkinsfile-basic.txt`

Set the environment variable value for your GitHub repo URL correctly.

Click Save.

Back in the main page for the job, select Build Now from the left hand menu to run your first build

Click on the completed build number to get more detailed build info.



Key options to select from the left hand menu list

- Console Output – view the console output from all stages of the pipeline job
- Pipeline overview – Allows you to view the output from each of the stages of the pipeline job
- Pipeline steps – Shows you the specific steps in each stage and how long it took to execute, as well as the corresponding output
- Workspaces – Allows you to navigate the workspace for that job after the completion of that current build. The workspace is the directory where Jenkins checks out the source code repository, runs the various commands for building, testing, etc in the various stages of the pipeline job, and creates build and test artifacts, as well as any other temporary files.

- Restart from stage – Allows you to run a new build by restarting from a particular stage of the current build.

This pipeline script replicates the main development workflow for the .NET CLI app that we performed earlier at the Bash shell / PowerShell through appropriately named stages.

The `deploy` stage simply executes the published DLL to validate it can be executed successfully.

6 Updating GitHub repo and running the build again

We will simulate the standard collaborative software development workflow by making a simple change to the main program of the app:
`src/PortableCliApp/Program.cs`

You can either edit this file directly using the MobaXTerm default editor or the built in CLI editor `nano`. If you wish to use `nano`, then from the project root folder, type:

```
nano src/PortableCliApp/Program.cs
```

You can also edit this file using an IDE (VS Code) in the project root folder on your local machine, and then transfer it to the EC2 Linux server by dragging and dropping into the navigation pane of the system

In this file, make a random change to one of the `Console.WriteLine` statements to produce different output.

Before creating a new commit incorporating this latest change and pushing it to the remote GitHub repo, you can first cache your username and PAT to avoid having to retype this at the shell:

```
git config --global credential.helper 'cache --timeout=30000'
```

```
git config --global credential.https://github.com.username your-username
```

Next, add these latest modifications to a new commit and push:

```
git commit -am "Made the first modification to app"
```

```
git push
```

After entering your PAT for this push, it should be cached with the credential helper and you should not need to re-enter it again for subsequent pushes.

Go back to the main page for this GitHub repo and verify that the latest commit has been successfully pushed to the `master` branch.

Back in the main page for the job on the Jenkin server, select Build Now from the left hand menu to run another build.
Click on the completed build number to get more detailed build info.

Check the Console Output and also the Pipeline overview to verify that the `deploy` stage executes the published DLL whose console output should reflect the latest updates to the GitHub repo that you just made.

You can repeat this process a few more times (making more random changes to `Program.cs`) to get a feel of the standard CI workflow with Jenkins, where the building, testing, publishing and execution of the app is performed with each update to the GitHub repo via a push.

After editing:
`src/PortableCliApp/Program.cs`

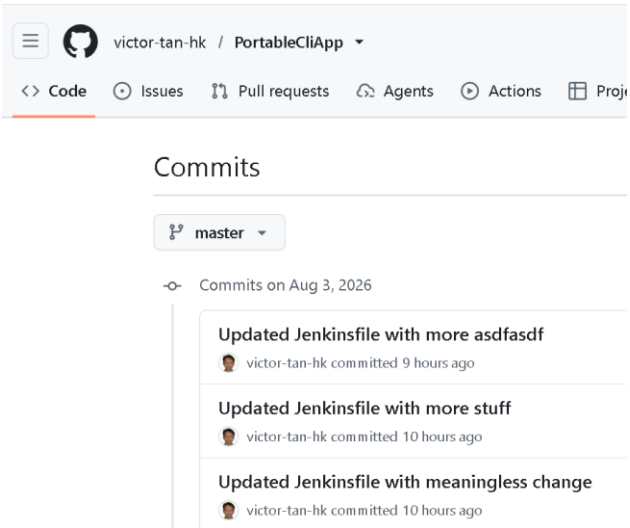
Add the latest modifications to a new commit and push:

```
git commit -am "Some suitable commit message"

git push
```

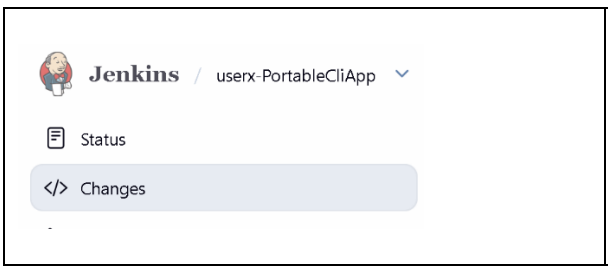
At any point in time in this development workflow, you can check the commit history of the `master` branch in the project root folder with:

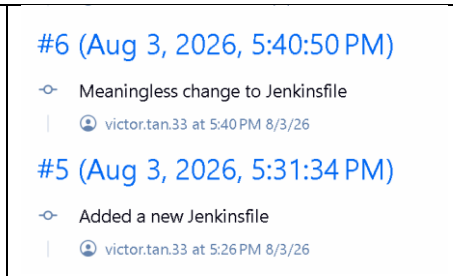
```
git log --oneline
```

and this history is also visible from the Commits page on the GitHub repo.

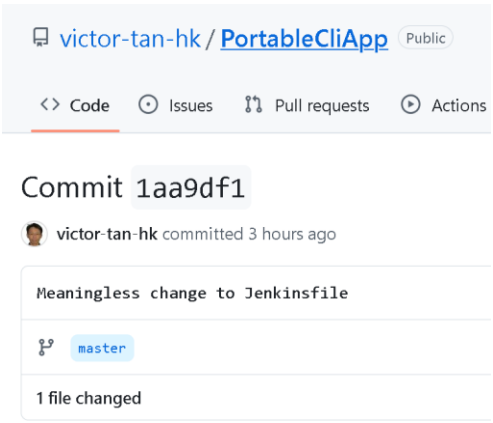


In the main page for the job, you can also select the Changes section in order to view the latest commits to the code base corresponding to the most recent builds on that job.



Clicking on any of these links will transition you to the GitHub page providing info for the commit at GitHub (assuming that your GitHub repo is available for public access).



7 Configuring the pipeline as a Jenkinsfile in SCM

Make sure you are in project root folder for the .NET Core CLI app in your home directory. If not, you can change to it with:

```
cd ~/githubprojects/PortableCliApp
```

We will create a new file here called `Jenkinsfile`.

You can either create this using the MobaXTerm default editor or the built in CLI editor `nano`. If you wish to use nano, then:

```
nano Jenkinsfile
```

You can also edit this file using an IDE (VS Code) in the project root folder on your local machine, and then transfer it to the EC2 Linux server by dragging and dropping into the navigation pane of the system

Copy and paste into this file the previous script from the `jenkinsfile collection` folder: `dotnetcli-jenkinsfile-basic.txt`
Then Save

After that, you can verify the content by typing:

```
less Jenkinsfile
```

Next, stage the new Jenkins file, add this to the commit and push:

```
git add .

git commit -am "Added a new Jenkinsfile"

git push
```

Go back to the GitHub page for this repo and verify that the Jenkinsfile is now checked in successfully into the project root folder.

Return back to the Configure page for the current Pipeline job, and in the Pipeline section, select Definition: Pipeline Script from SCM and fill in the Repository URL with the HTTPS URL of your GitHub Repo.

The most important part is the script path: which specifies the path and the name of the Jenkinsfile. Here, we retain the default name (Jenkinsfile) and the default location (the root folder of the Git project). For more complex projects, you might want to provide a different name and different location, in which case, you must explicitly specify it here.

Click Save and Run a Build Now again with this new configuration, whereby the build is now being executed based on the pipeline script in the Jenkinsfile in the SCM.
Verify that it completes successfully as before where the stages in the Pipeline Overview and the console output from each stage is identical as before.

Let's make a random addition to the Jenkinsfile in the project root folder using either the CLI editor `nano`, MobaXTerm Editor or VS Code and copying over to the EC2 server.

Put in this snippet between the `clean` and `restore` stages:

```
    stage('Dummy') {
      steps {
        echo 'Some random statement to demonstrate change'
      }
    }
```

Save this.

Commit this change and push the new commit to the remote repo:

```
git commit -am "Meaningless change to Jenkinsfile"

git push
```

Check for this latest change in the GitHub repo.
Back in Jenkins, run another Build on this job and verify that the Console Output and Pipeline Overview shows this `Dummy` stage and the output from the `echo` statement in it.

By incorporating the Jenkinsfile in the project root, we ensure that it is maintained in the GitHub repo along with the app code base. We can therefore perform version tracking on changes to it in the same way that we do for the app source code and other configuration files

This means as the project evolves over its lifetime and changes are made to the Jenkinsfile to control how the build is being performed, we can keep track of how the build was actually performed at a point in time and the author of the associated changes to that Jenkinsfile.

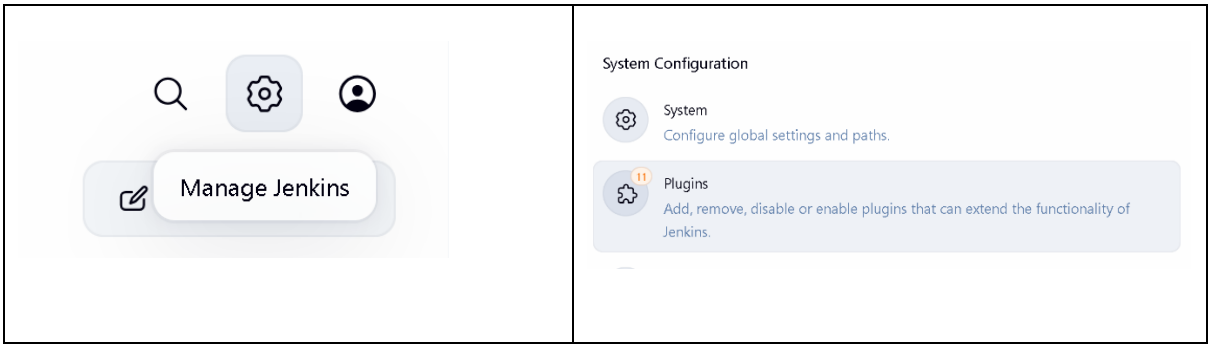
Remember that this is considered best practice for specifying a pipeline script, and you should always endeavour to do this in all your projects.

8 Installing plugins and generating and visualizing test result trends

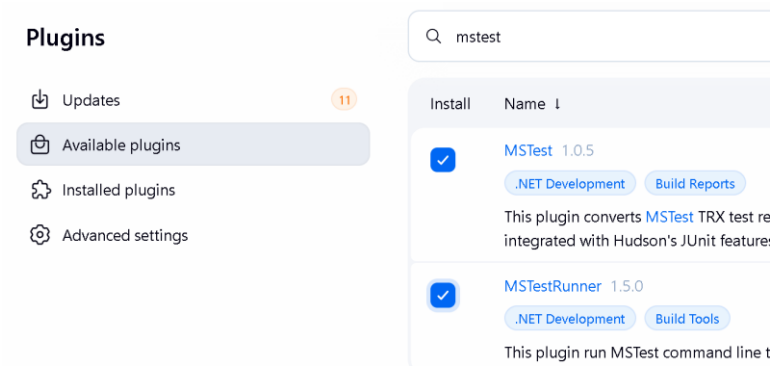
For visualization of test trend results, we will need to install the MSTest and MSTestRunner plugins. Remember that plugins perform most of the additional functionality required in a Jenkins job beyond the core functionality that ships with a fresh Jenkins server install.

This installation only needs to be done once by the admin account

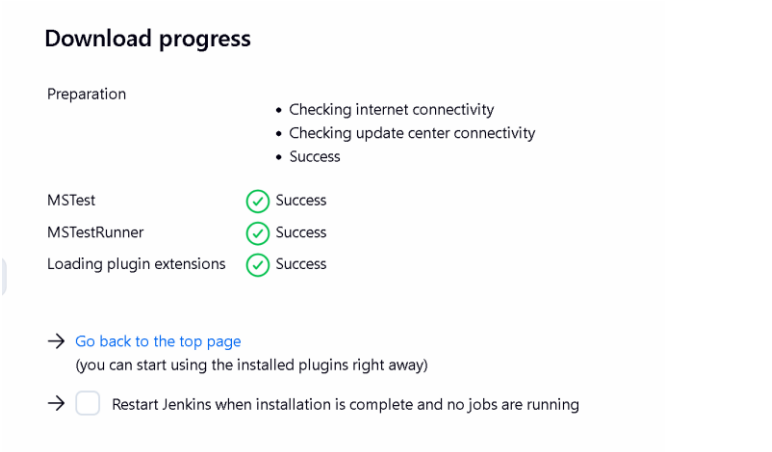
Go to Manage Jenkins (upper right hand corner), select Plugins.



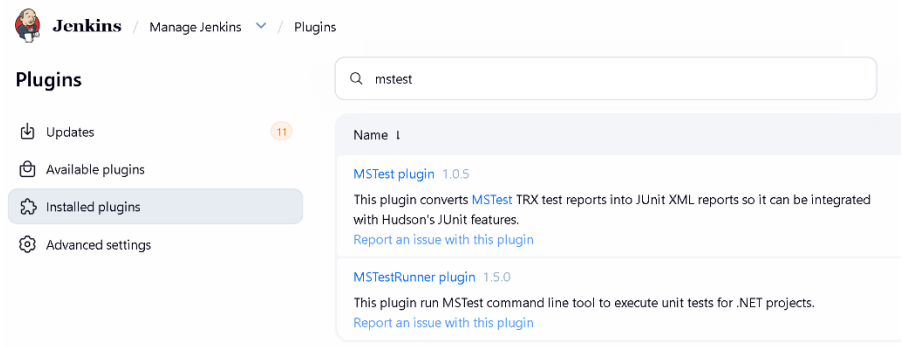
Search for MSTest and MSTestRunner in the Available Plugins and select them and click Install.



Select Restart Jenkins when Installation is complete.



Jenkins will restart and you will need to sign in again.
Check that the plugins are properly installed by checking in the Installed Plugins section.



You can then return to the main page of your pipeline job.

We will further update the Jenkinsfile to generate test results during the testing stage in MS TRX format, which is a machine-readable XML-based test-report format. These are stored in a predictable workspace directory for Jenkins to locate in the `post` section in the `always` step.
Here, we publish every TRX test-result file generated under the specified directory that we used earlier.
The Jenkins MSTest plugin converts the TRX results into internal test-result representation used by Jenkins so that it can be visualized appropriate in the UI.

Make in a further change to the Jenkinsfile in the project root folder using either the CLI editor `nano`, MobaXTerm Editor or editing in VS Code and copying over to the EC2 server.

Copy and paste into this file this script from the `jenkinsfile collection` folder: `dotnetcli-jenkinsfile-visualize-tests.txt`

Save this.

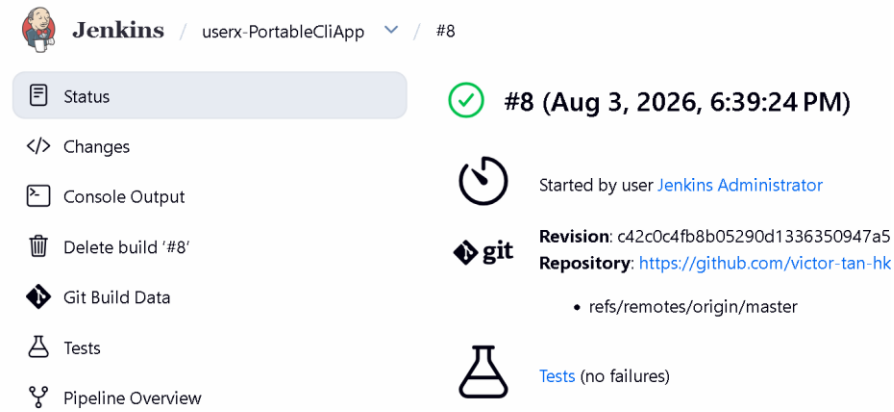
Commit this change and push the new commit to the remote repo:

```
git commit -am "Updated Jenkinsfile for visualization of test results"

git push
```

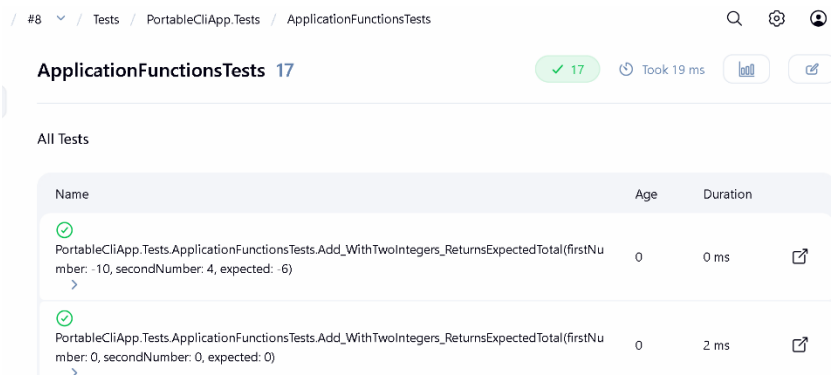
Check for this latest change in the GitHub repo.
Back in Jenkins, run another Build on this job.

When this completes, you should be able to see a test trend overview on the main page of the job.

Within the main page for that build, you should be able to see additional links you can access regarding the tests.



Clicking and following through on these links allows you to view specifics of all the unite tests that were run from the main test file: `ApplicationFunctionsTests.cs`



Run a few more builds and see the Test Trends on the main page populate.

Simulate an error in the unit tests by intentionally introducing some errors into the application source code by editing: `src/PortableCliApp/ApplicationFunction.cs`

You can do this by using the built-in Linux nano editor

```
nano src/PortableCliApp/ApplicationFunction.cs
```

or using the default MobaXTerm editor or editing in VS Code and copying over to the EC2 server.

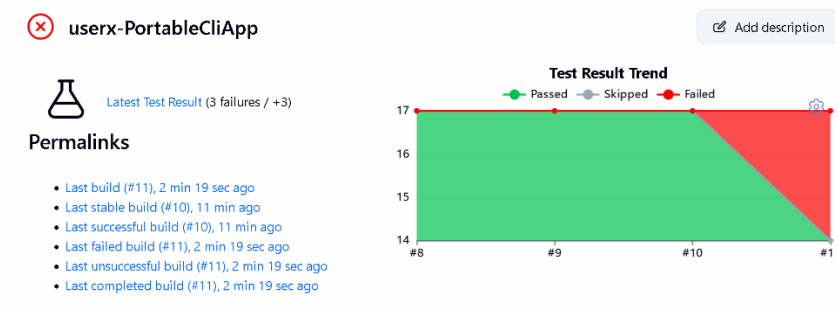
As usual commit the changes and push the new commit to the remote repo:

```
git commit -am "Simulating a stupid error !"
```

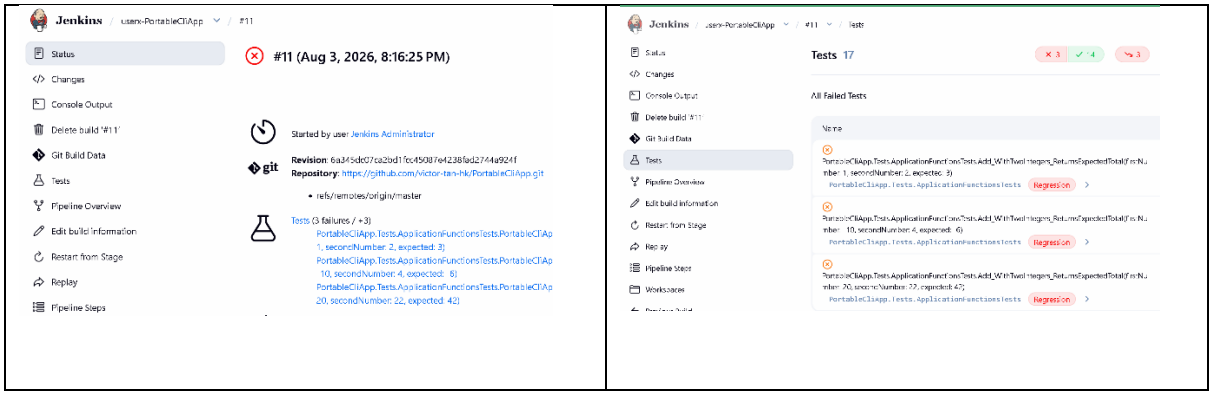
```
git push
```

Run the build again on this job in Jenkins. This time you should see the error reported in the Console Output for that particular build attempt.

The main page for the job should now indicate the build failure in the Test Result Trend (Refresh the page if it does not show up immediately) and also provide links to the various builds with various statuses (stable, successful, failed, unsuccessful, etc).



In the main page for the build, you should also be able to access information about the test failures:



Correct the error in the unit tests by correcting the error you had intentionally introduced into the application source code by editing: `src/PortableCliApp/ApplicationFunction.cs`

You can do this by using the built-in Linux nano editor

```
nano src/PortableCliApp/ApplicationFunction.cs
```

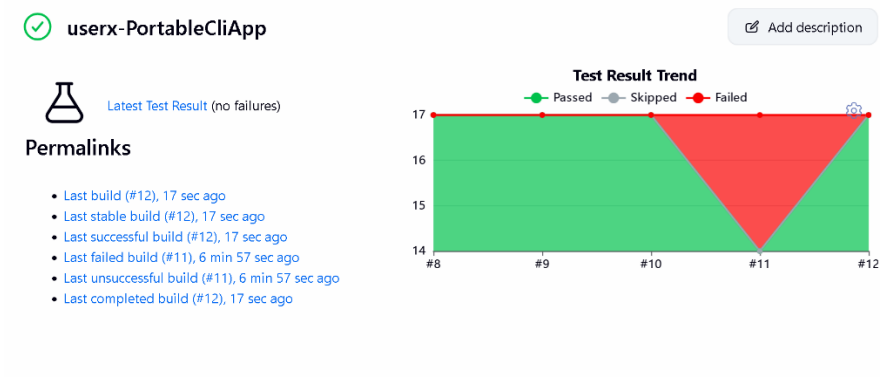
or using the default MobaXTerm editor or editing in VS Code and copying over to the EC2 server. Save and exit.

As usual commit the changes and push the new commit to the remote repo:

```
git commit -am "Corrected my stupid error !"
```

```
git push
```

Run the build again. This time the build should be successful, and the various info on the main page for the job should reflect that as well (refresh the page if the Test result trend does not seem to change with the latest build result).



9 Archiving build and test artifacts

Make in a further change to the Jenkinsfile in the project root folder using either the CLI editor `nano`, MobaXTerm Editor or editing in VS Code and copying over to the EC2 server.

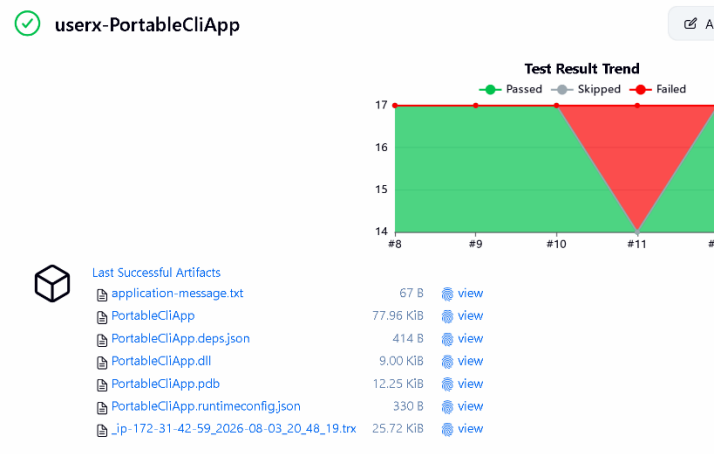
Copy and paste into this file this script from the `jenkinsfile collection` folder: `dotnetcli-jenkinsfile-archive.txt`

Save this.

Commit this change and push the new commit to the remote repo:

```
git commit -am "Updated Jenkinsfile for archival of artifacts"

git push
```

If you now refresh the main page for the job (or the main page for the build), you should see links for you to download the various build / test artifacts.



Archiving artifacts from the build / test stages is very useful if the Jenkins server is running on a remote server, as it allows you to download these artifacts to your local machine or to the production server for further use.

10 Configuring periodic builds and SCM polling

At this point of time, we are manually performing our builds by clicking on the Build Now item in the main page for each job. This is not practical or ideal for real life projects, where team members may be pushing new changes to the central repo at variable times throughout the day.

Jenkins includes what is known as build triggers, which allows a build for a job to be triggered automatically when a specific event occurs. The most common situations would be:

a) Periodically performing the build, for e.g. once every hour, or every 6 hours, or every day

b) Checking the remote repo periodically for any updates (this is known as polling SCM) and then performing the build if there are any updates since the last poll. This is nearly the same as periodic builds, except that periodic builds will always occurs, regardless of whether there are any updates to the remote repo.

c) When a specific event occurs (for e.g. a new commit is pushed to some branch on a remote repo, a pull request is opened to merge a feature branch back into the master branch, etc), the remote repo sends a signal (typically a HTTP POST request) to the Jenkins server to trigger

the build. This feature is known as a webhook and is available with most Git cloud hosting services (GitHub, BitBucket, etc)

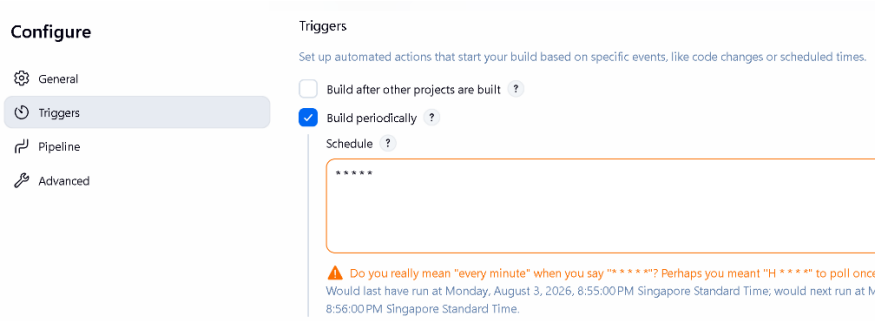
For the first 2 approaches, the scheduling is based on the CRON job format from Linux. This is commonly used to schedule the automated running of scripts to perform system maintenance tasks periodically.

An example of the CRON expression to run a job every minute

The Jenkins CRON expression is nearly similar to the Linux CRON syntax with some minor differences:

From the main job page, select Configure, and go Triggers, select Build Periodically and enter the CRON expression to trigger a build on the pipeline every minute:

* * * * *



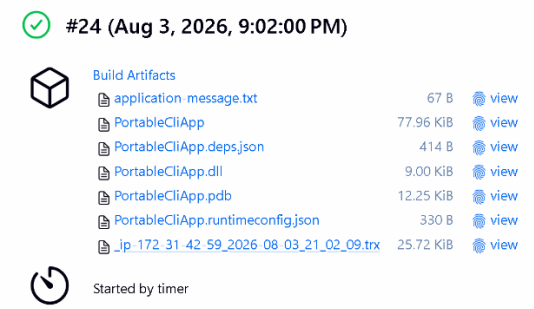
A warning will be issued regarding this CRON expression as it is not realistic to trigger a build every minute (we are only doing this for demo purposes in this workshop).

You can choose the CRON expression for the suitable interval of your choice in your real-life projects, using an AI tool to generate it for the specified interval.

Click Save and run the first build. Notice after the first build, a new build will be triggered every minute. Watch and monitor for this.

Notice that a build will automatically be triggered every minute - **EVEN IF NO** updates have being made to the remote repo. You may have to refresh the main job page to see this periodical build being triggered every minute.

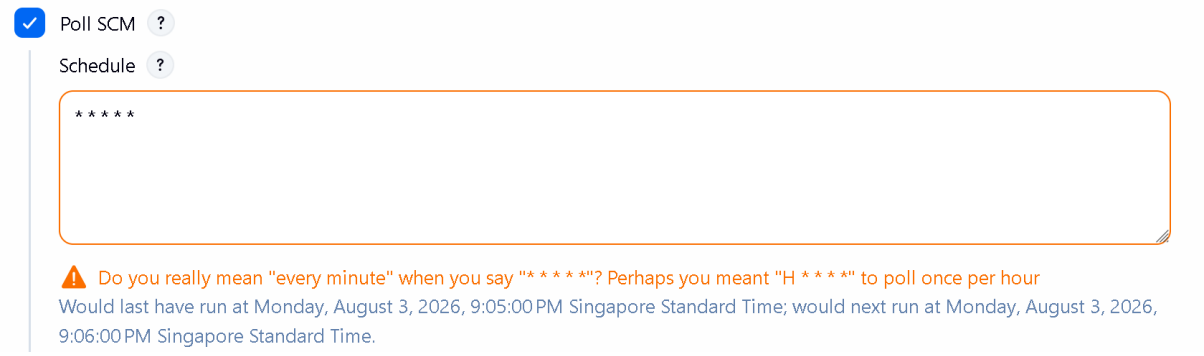
The build main page also explicitly indicates that the changes were started by the Jenkins timer (rather than manually by a user, as in the case of the previous builds).



Next, we will configure Polling SCM, which means checking the remote repo periodically for any updates (this is known as polling SCM) and then performing the build if there are any updates since the last poll. If there are no updates, no build is run.

From the main job page, select Configure, and go Triggers, select Poll SCM and enter the same previous CRON expression to trigger a build on the pipeline every minute:

```
* * * * *
```



Click Save.

Go back to the main page for the job and wait for 3 - 5 minutes. Notice that unlike before, no builds are triggered.

Make in a meaningless change to the Jenkinsfile in the project root folder using either the CLI editor `nano`, MobaXTerm Editor or editing in VS Code and copying over to the EC2 server.
For e.g. enter an additional space or new line in an appropriate location in this file.

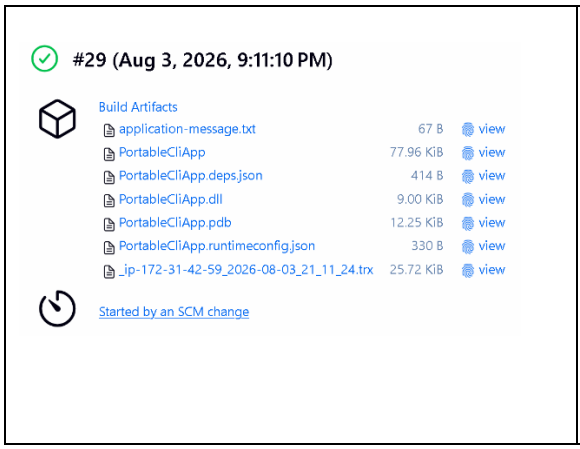
Save this.

Commit this change and push the new commit to the remote repo:

```
git commit -am "Updated Jenkinsfile with meaningless change"

git push
```

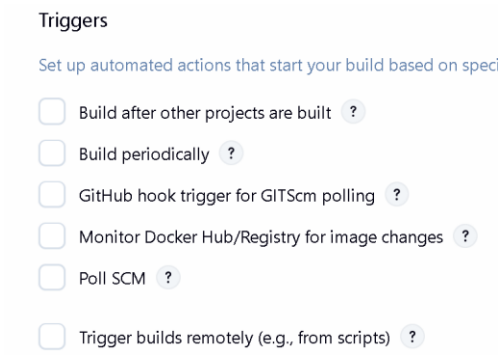
Go back to the main page for the job and wait for a minute. Now a build will be triggered because the polling of the GitHub repo indicates that there has been change in it since the last poll.

Notice as well that the build page clearly shows the build was triggered by a SCM change, and the Polling Log provides the info on the polling operation.



Now leave it again for 2-3 minutes and notice that no further builds are triggered.

When you are done verifying both these features (periodic builds and SCM polling), go to Configure and disable all Triggers in the Triggers section and Save the job again. We will enable them later if we need them in subsequent lab sessions.



11 Configuring Webhooks with GitHub

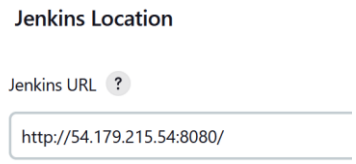
In the previous lab, we looked at 2 different build triggers: Periodical builds and Poll SCM when working with remote Git repos. However, the most popular and useful trigger for interacting with remote Git repos is webhooks.

When a specific event occurs (for e.g. a new commit is pushed to some branch on a remote repo, a pull request is opened to merge a feature branch back into the master branch, etc), the remote repo sends a signal (typically a HTTP POST request) to the Jenkins server to trigger the build. This feature is known as a webhook and is available for use with repos on the main Git cloud hosting services (GitHub, BitBucket, etc)

At the Jenkins main dashboard, go to Manage Jenkins -> System -> Jenkins Location

The Jenkins URL is effectively the canonical public base URL for the controller, which is used for generating redirects when working with WebHooks. The Jenkins URL should already be set to the

public IP address of the AWS EC2 instance running the Jenkin Server (check the address bar to verify this for sure) as well as the designated port:

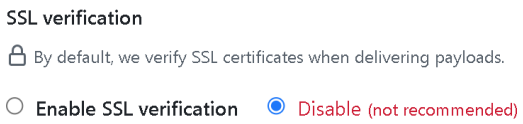


To setup a WebHook for the GitHub repo `PortableCliApp`,
Go the repo and Click Settings.
In the left sidebar, click Webhooks.
Click Add webhook.
For the Payload URL, enter:
http://*IP-address-JenkinsServer*:*port*/github-webhook/

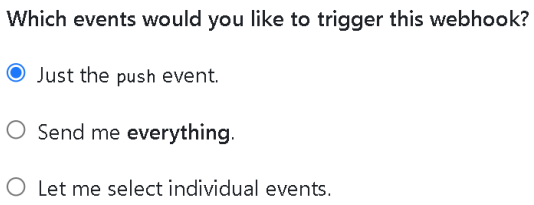
for e.g. **http://56.68.71.41:9090/github-webhook/**

Make sure you type the webhook URL with an ending / or else an error will occur when processing the HTTP POST request sent from GitHub on the Jenkins server end.
In the Content type select: `application/json`
Leave the Secret field empty.

Disable SSL verification as we have not set up HTTPS for our Jenkins server.

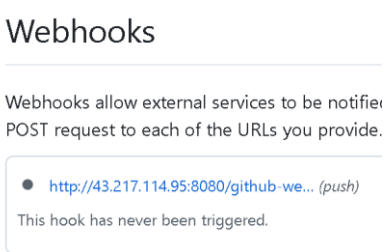


For events to trigger the webhook, we stick to the default of just the push event.

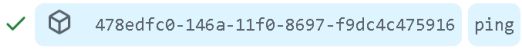


For real life projects, you can click Let me select individual events, and decide which other events on the repo will trigger the webhook. Other common events besides a push are branch creation, commit comments, discussion comments, issues, pull requests and reviews and comments, etc.

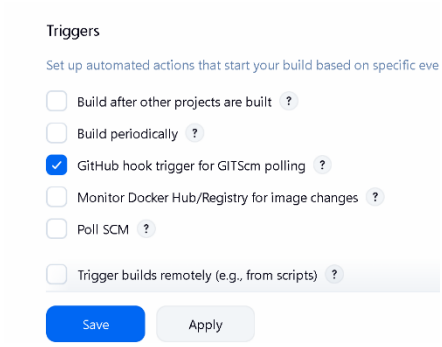
Click Add Webhook.



Click on the webhook entry above and select Recent Deliveries. This will show the results of GitHub sending a test ping to the Jenkins server at the specified URL and if you click on the … menu at the end of the ping message, you should see the test request sent out and the successful response (HTTP code 200 Ok) received in return.



In the main page for the job, select Configure, Triggers -> GitHub hook trigger for GitScm polling.



Click Save.

Wait for a few minutes. Notice that nothing happens.

Make a meaningless change to the Jenkinsfile in the project root folder using either the CLI editor `nano`, MobaXTerm Editor or editing in VS Code and copying over to the EC2 server.
For e.g. enter an additional space or new line in an appropriate location in this file.

Save this.

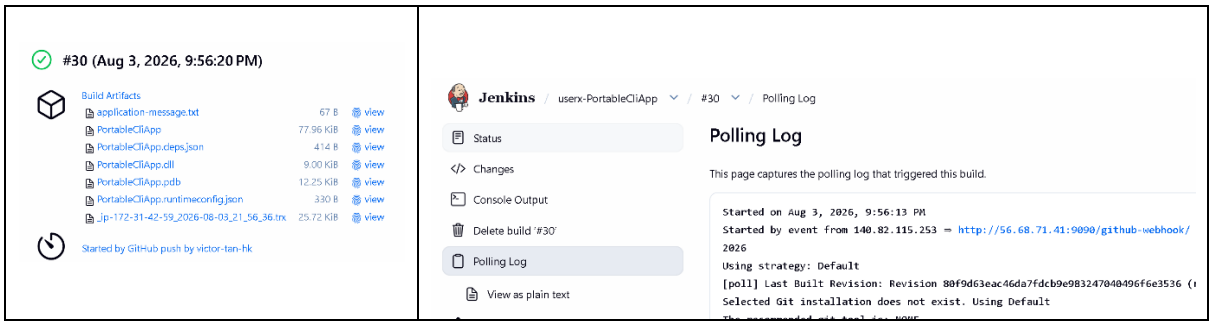
Commit this change and push the new commit to the remote repo:

```
git commit -am "Updated Jenkinsfile with more stuff"

git push
```

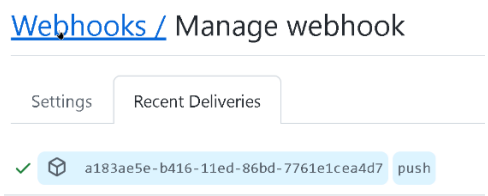
Return to the main job page for in the Jenkins UI, and keep your eye on the Build History

Notice now that a build is automatically triggered as a result of the webhook that you have setup. You may have to refresh the page to see this happen.

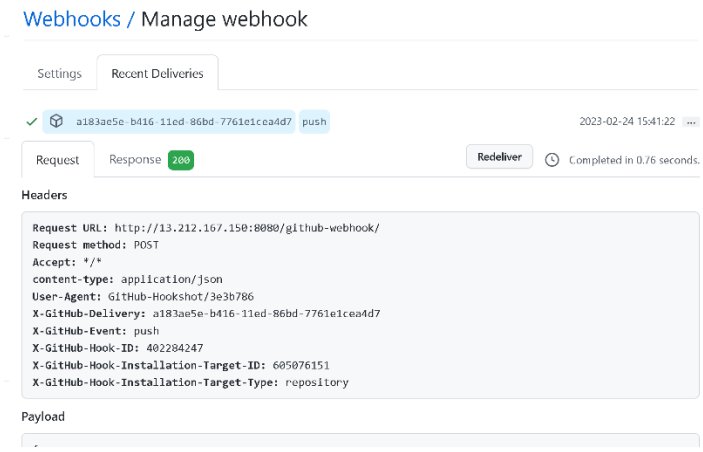
Notice as well that the main page for build clearly shows the build was triggered by a GitHub Push, and the Polling Log provides the info on the push operation itself.



The GitHub Webhooks page for that particular WebHook should also correspondingly show the HTTP POST request that was sent out to the Jenkins server in order to trigger this build in the Recent Deliveries tab.



You can drill down into the request and response by clicking on the ellipsis menu … at the end of the entry to perform trouble shooting if necessary.



You can continue to make more random changes, make a commit to incorporate these changes and push to your GitHub repo to test the webhook triggering the build of your pipeline job:

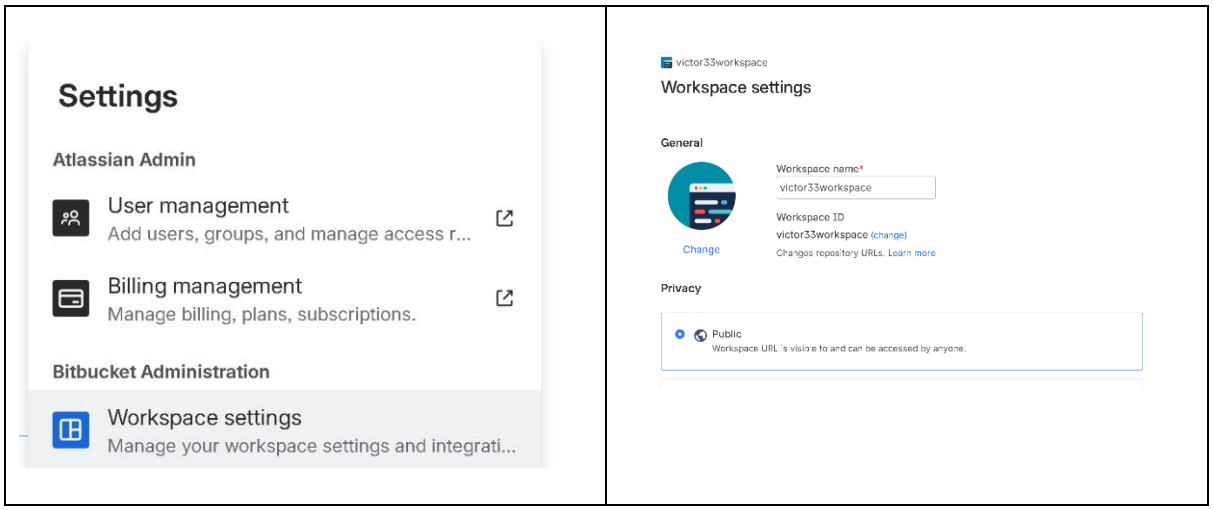
```
git commit -am "Updated Jenkinsfile with more stuff"

git push
```

12 Creating a BitBucket repo for the project

We will now create a new empty remote repo on BitBucket and push the contents of a local repo to populate it, the same way that we did for our GitHub repo.

Login to your Bitbucket account. You may have to create a new workspace if you are working with a new Bitbucket account.

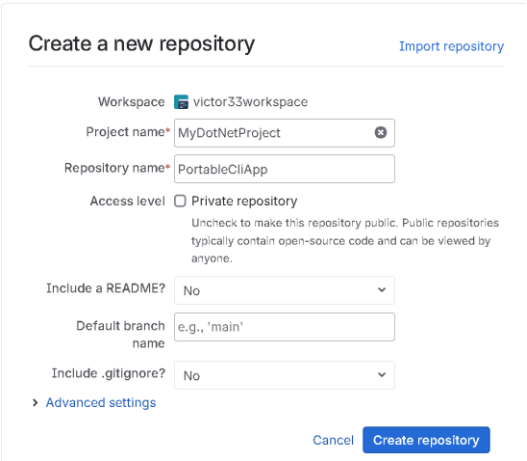
Go to your Workspace Settings from the main workspace page, and make sure it is set to be Public.



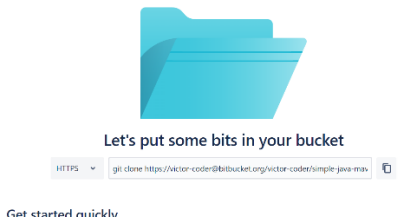
Go through the process of creating a new repository, but create a bare repository with no content to facilitate the process of pushing the contents of our local repository to it:

Enter the values for the following fields.
Project Name: `MyDotNetProject`
Repository Name: `PortableCliApp`

Make sure this is **NOT** a private repository, and **DON'T** include either a readme file or .gitignore. This is to ensure that the repository is a bare repository that we can push new content into from our local repo.



When you are done specifying the values for the fields as shown above, click Create Repository. You will be transitioned to the Source view for the newly created repo, where some instructions will be provided on how to get started.



Click on the Copy button to copy the URL (e.g. https://xxx@bitbucket.org/yyy/simple-java-maven-app.git) for this new remote repo to an empty document. We will refer to this URL as *`remote-url`* in the commands to follow.

In the home directory of your assigned user account, there is a `bitbucketprojects` folder, which in turn contains a `PortableCliApp` folder. This is an exact copy of the same .NET CLI project that we were working with earlier in the `githubprojects` folder

In the Bash shell, change to this folder with:

```
cd ~/bitbucketprojects/PortableCliApp
```

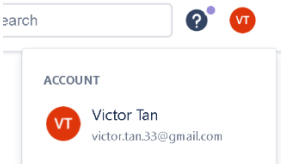
As we did earlier, initialize this as a new Git repo with:

```
git init
```

```
git add .
```

```
git status
```

Before creating the first commit in this newly initialized repository, you need provide an identify for the individual creating the commit (the commit author). Since we are going to be pushing our local repo to populate the BitBucket repo, we will use our BitBucket identity (name and email). This is accessible from the upper right hand corner of the page.



and note down the name and email address there. Use them in the configuration commands below:

```
git config --local user.name "your name"
```

```
git config --local user.email "your email address"
```

We are configuring these details at the local level so that they apply only to this repository .

We will leave the previous settings for our GitHub account at `global` level, so we can reuse them for the other remaining projects that we will host at GitHub.

Double check that these configuration variable values have been set properly at the local level, which will be in the file `.git\config` in the current working directory

```
git config --local --list
```

Then we can create our first commit.

```
git commit -m "Added all project files in first commit"
```

```
git status
```

Next, we associate the remote repo URL with our newly initialized local repo with:

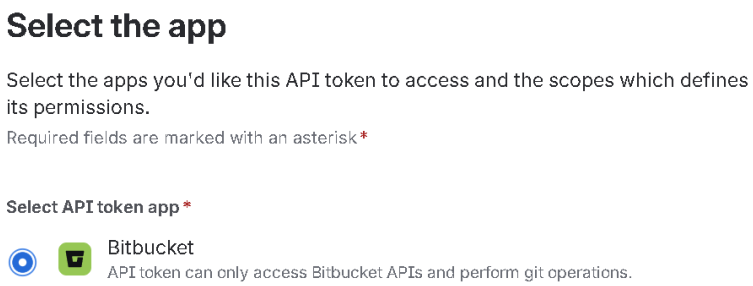
```
git remote add origin remote-url
```

Check that the origin handle is set to point to the correct repo URL with:

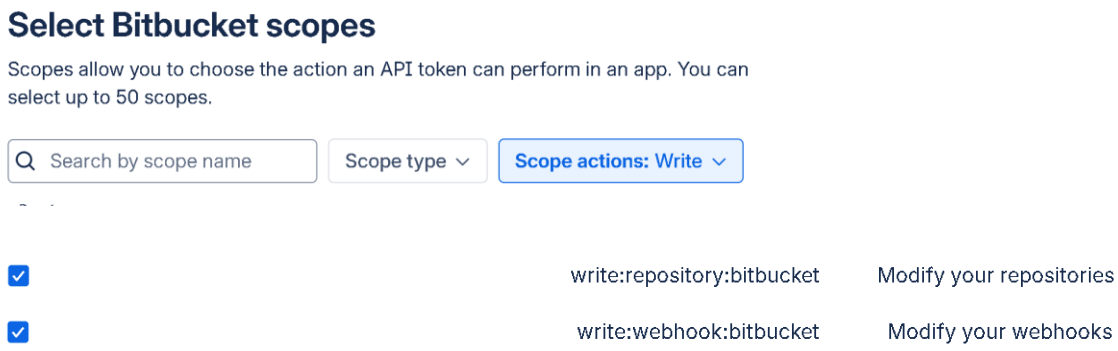
```
git remote --verbose
```

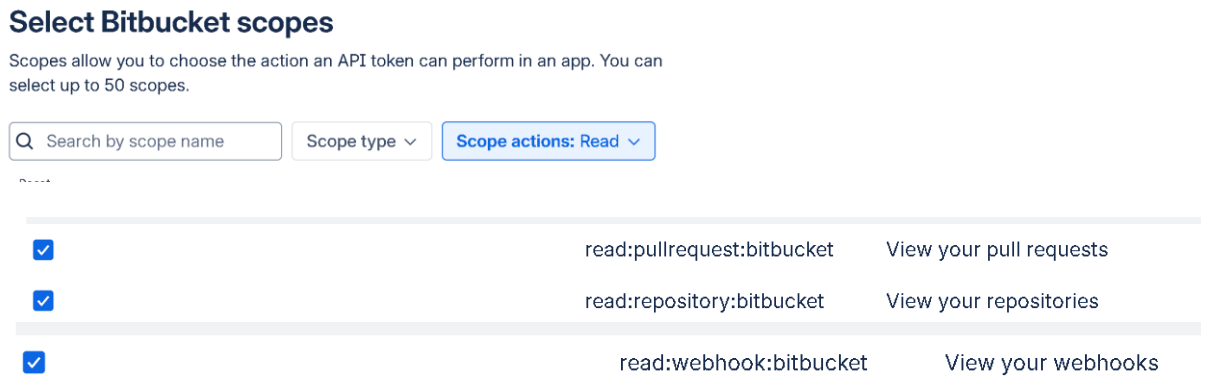
Before you can push the contents of the newly initialized repo to your remote BitBucket repo, you must first obtain an API token. This is conceptually similar to classic personal access token (PAT) that we used in GitHub repo.

Create the API token and give it a suitable name and select use of the token with BitBucket

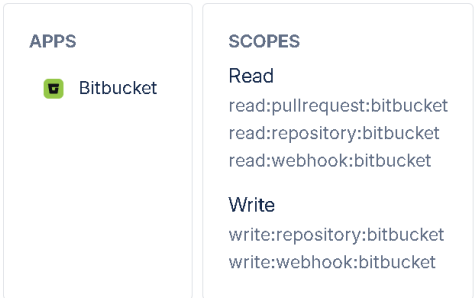


Then select the scope type for the following Scope Actions: Write and Read.





Click Next and verify that you have all the right scopes for Read and Write.



Click Create Token.
When the API Token is shown in the dialog box, make sure you copy it down somewhere where you can access it before closing it.

Finally, push the entire contents of the local repo (including all its branches) to the remote repo with:

```
git push -u origin --all
```

You will be prompted for your app password:

```
Password for 'https://xxxxx@bitbucket.org':
```

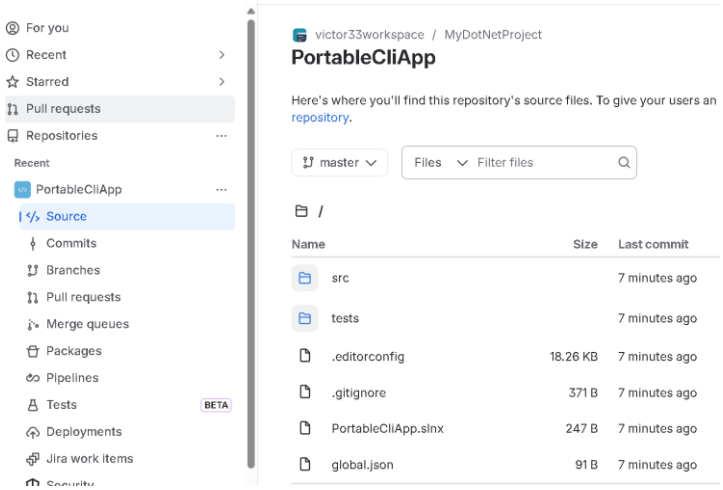
You can copy and paste the API Token password, but be very careful to do this slowly and correctly.

Assuming you have entered the correct app password, the command should succeed with appropriate success messages in the Bash shell.

```
Enumerating objects: 39, done.
….
…..
* [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

Return back to the browser and refresh it to ensure that you have the latest view with the uploaded repo content. Check through the Source, Commits and Branches main view to verify that these mirrors

the status of the local repo that you just pushed up into it. This may take some time to complete correctly.



Back in the Bash shell, type the following commands to verify that the remote tracking branches have been set up and that the local and upstream master are both in sync with each other.

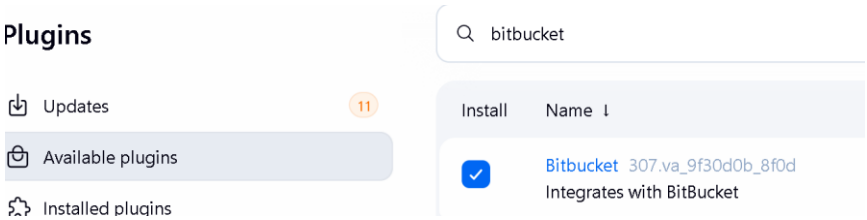
```
git remote show origin

git status
```

13 Configuring Webhooks with BitBucket

We are now going to configure a WebHook for our BitBucket repo, which is conceptually the same as what we did for GitHub earlier.

Only one person with admin access needs to perform the plugin installation detailed below.

From the main Jenkins Dashboard, Manage Jenkins -> Plugins -> Available Plugins, look for the following Bitbucket plugin ONLY and select it.



Select Install and click Restart Jenkins when installation is complete and no jobs are running once the download is complete.
You may need to refresh the webpage to validate when the installation is complete, and login again.

As before, after the restart of the Jenkins server, go to the Installed Plugins section to verify that this single plugin has been successfully installed.

From the main Jenkins dashboard, create a new Item and name it:

`userx-Bitbucket-PortableCliApp`

Make this a Pipeline and select Ok.

You will be transitioned to the Configure section.
Select the Pipeline section.

Copy and paste into the Script window this script from the `jenkinsfile collection` folder:
`dotnetcli-jenkinsfile-basic.txt`

For this example, we do not need to create a complete Jenkinsfile and check that in with our application code based in the BitBucket repo, as we have done previously. We can define the pipeline script directly in the Script window and this will work as well, since we are only interested in testing the triggering of the build through a Bitbucket webhook.

Set the environment variable value for your BitBucket repo URL correctly in the script.

Click Save.

Back in the main page for the job, select Build Now from the left hand menu to run your first build

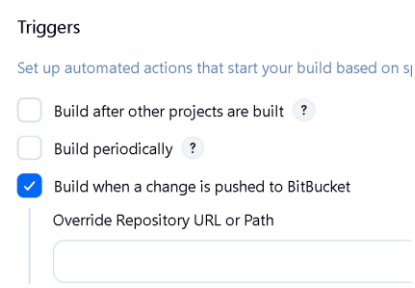
13.1 Exercise: Bitbucket WebHook configuration

As an exercise, setup the configuration for a BitBucket Webhook and test it by pushing new commits from your local repo.

Some sample guides to set up Bitbucket Webhook are shown below:
https://support.atlassian.com/bitbucket-cloud/docs/manage-webhooks/
https://magefan.com/blog/add-webhook-in-bitbucket

USEFUL TIPS:

- The WebHooks link is within the WorkFlow link in the Repository Settings
- The webhook URL that you configure on BitBucket should be in this form:
  `http://IP-address:port/bitbucket-hook/`
- You should skip certificate verification in the Webhook configuration as the Jenkins server for our workshop is not secured with SSL/TLS
- You can select Repository Push as your trigger when configuring the WebHook
- In the View Requests log section, you can select to archive a history of requests for inspection
- You can then click on view request logs / Load new requests to see the history of requests send from BitBucket to the Jenkins service in response to the configured events (push commits to the remote repo). Unlike GitHub, Bitbucket does not perform an initial ping test request: in order to test out the webhook functionality, you will have to push a new commit into your remote Bitbucket repo.
- You should ensure that the trigger `Build when a change is pushed to Bitbucket` is selected in the Triggers section of the Configuration for your pipeline job on Jenkins. You don't have to provide any other configuration values.



For a new commit to test out the webhook, just create a new file in the project root folder, for e.g:

`readme.txt`

and populate it with some random content using either the CLI editor `nano`, MobaXTerm Editor or editing in VS Code and copying over to the EC2 server.

Save and add this file to the commit.

```
git add .
```

```
git commit -m "Added readme.txt"
```

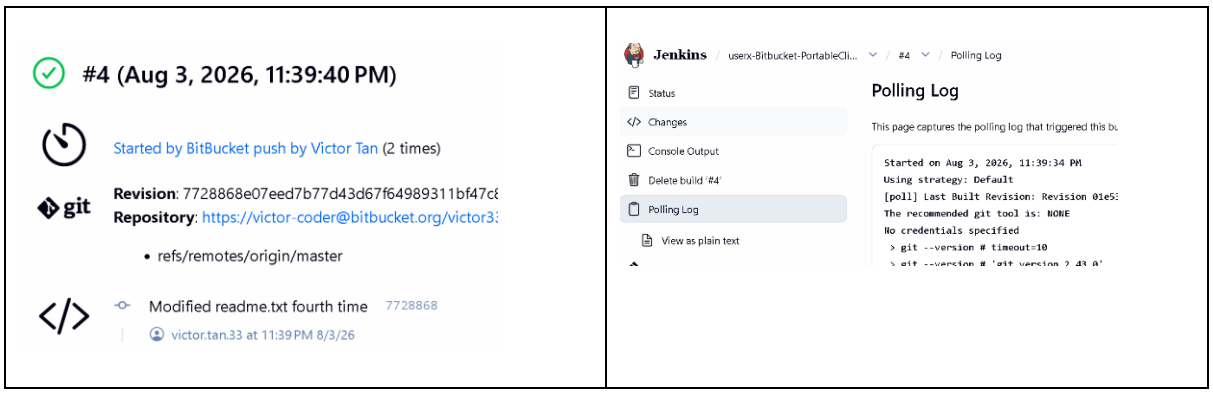
Then push to the BitBucket repo with:

```
git push
```

Return to the main job page for in the Jenkins UI, and keep your eye on the Build History

Notice now that a build is automatically triggered as a result of the webhook that you have setup. You may have to refresh the page to see this happen.

As in the case of the GitHub Webhook, notice here as well that the main page for the triggered build clearly shows the build was triggered by a Bitbucket Push, and the Polling Log provides the info on the push operation itself.



You can continue to make further changes, commit these changes and push them to your remote repo with:

```
git commit -am "Modified readme.txt first time"
```

```
git push
```

Monitor again that the build of the pipeline job is automatically triggered by the webhook for each push operation.

14 END